SPARC-PM

Software Design Document

Space Weather Risk Classification & Predictive Maintenance Engine

Design coverage: principles, architecture, interface design, implementation evidence, and key decisions

Prepared for project review

Repository baseline: Streamlit mission-control dashboard
Date: 11 September 2026
Scope: current implementation plus clearly marked production follow-ups

The design is intentionally operational: telemetry is transformed into explainable risk, crew guidance, fleet actions, and auditable commands.

1. Design principles applied

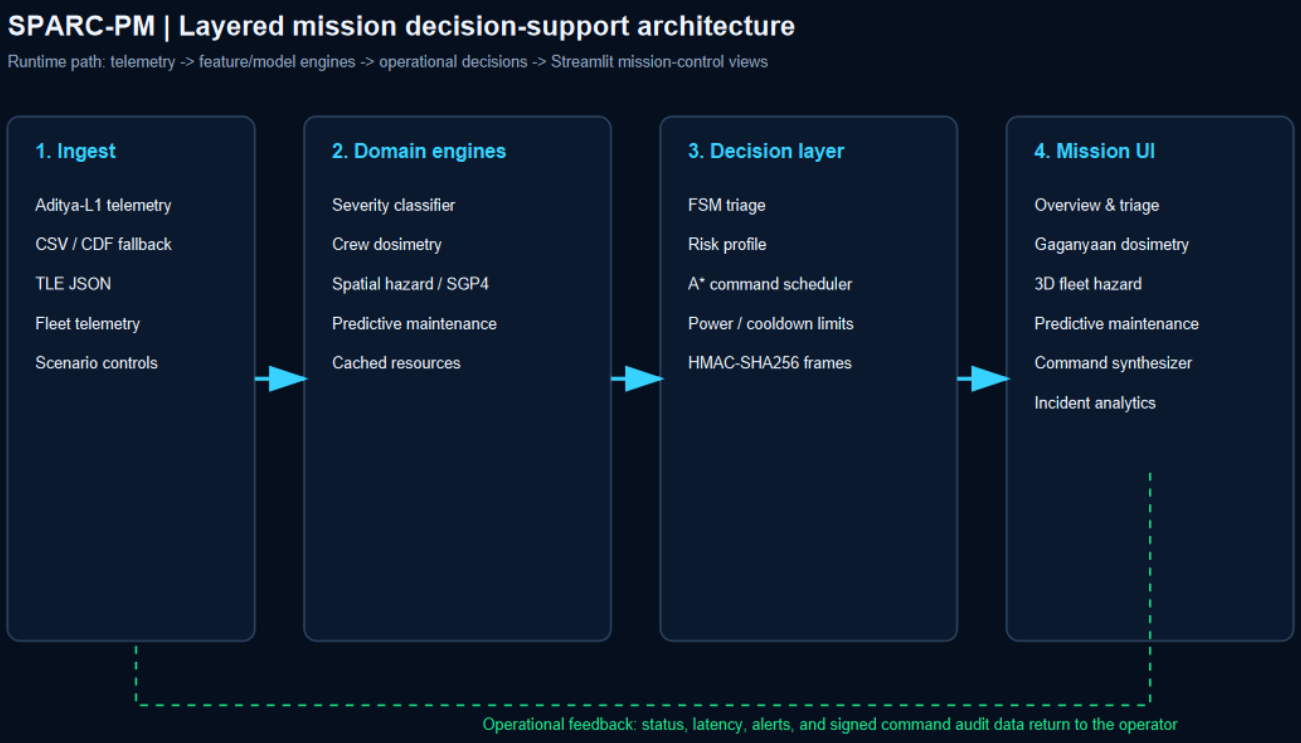
SPARC-PM separates scientific computation from operator presentation. This keeps the dashboard readable while allowing each algorithm to be tested and replaced independently.

Abstraction	The dashboard consumes engine-level reports rather than reimplementing physics, model inference, or scheduling logic in UI code.
Modularity	Severity, dosimetry, spatial hazard, maintenance, and scheduling are separate modules with focused responsibilities.
Cohesion	Each engine owns its domain constants, transformations, thresholds, and output contract; related behavior stays together.
Low coupling	The UI communicates through dictionaries/data frames and cached constructors, so an engine can change without changing all six views.
Operational safety	Triage actions, power budgets, cooldowns, cryptographic signatures, and a manual override boundary are explicit concerns.

Cheap validation check

A change to a model engine should be testable without launching Streamlit. The current module boundaries support that check; the remaining improvement is adding a formal automated test suite for the contracts.

2. High-level architecture



Architecture style: layered pipeline with a decision boundary

The layered style was chosen because the system has a directional flow from sensor data to decisions, while the decision layer must remain visibly distinct from presentation. It makes ownership, testing boundaries, and failure containment easy to discuss in a review.

Runtime sequence

1. Ingest telemetry and TLE/fleet data.
2. Normalize and derive physical features.
3. Run specialist engines and cache stable resources.
4. Apply FSM triage and A* prioritization.
5. Present six operator views and signed/auditable command results.

3. Implementation evidence

The following excerpts show how the architecture is expressed in code.

```
@st.cache_resource
def load_spatial_engine():
    return SpatialHazardEngine()

@st.cache_resource
def load_dosimetry_engine():
    return CrewDosimetryEngine()

@st.cache_resource
def load_action_scheduler():
    return AStarActionScheduler()
```

Resource caching reduces repeated model and fleet initialization during Streamlit reruns. It is a presentation-layer optimization that does not leak into domain logic.

```
telemetry_packet = {
    "proton_flux_pfu": sim_flux,
    "proton_speed_kms": sim_speed,
    "mag_bz_field": sim_bz,
    "proton_density_cm3": sim_density,
    "solar_wind_dyn_pressure_npa": sim_pressure,
}

severity_report = clf.predict_single(telemetry_packet)
hazard_report = spatial_eng.evaluate_storm_hazards(
    solar_wind_speed_kms=sim_speed,
    bz_field_nt=sim_bz,
    proton_flux_pfu=sim_flux,
)
```

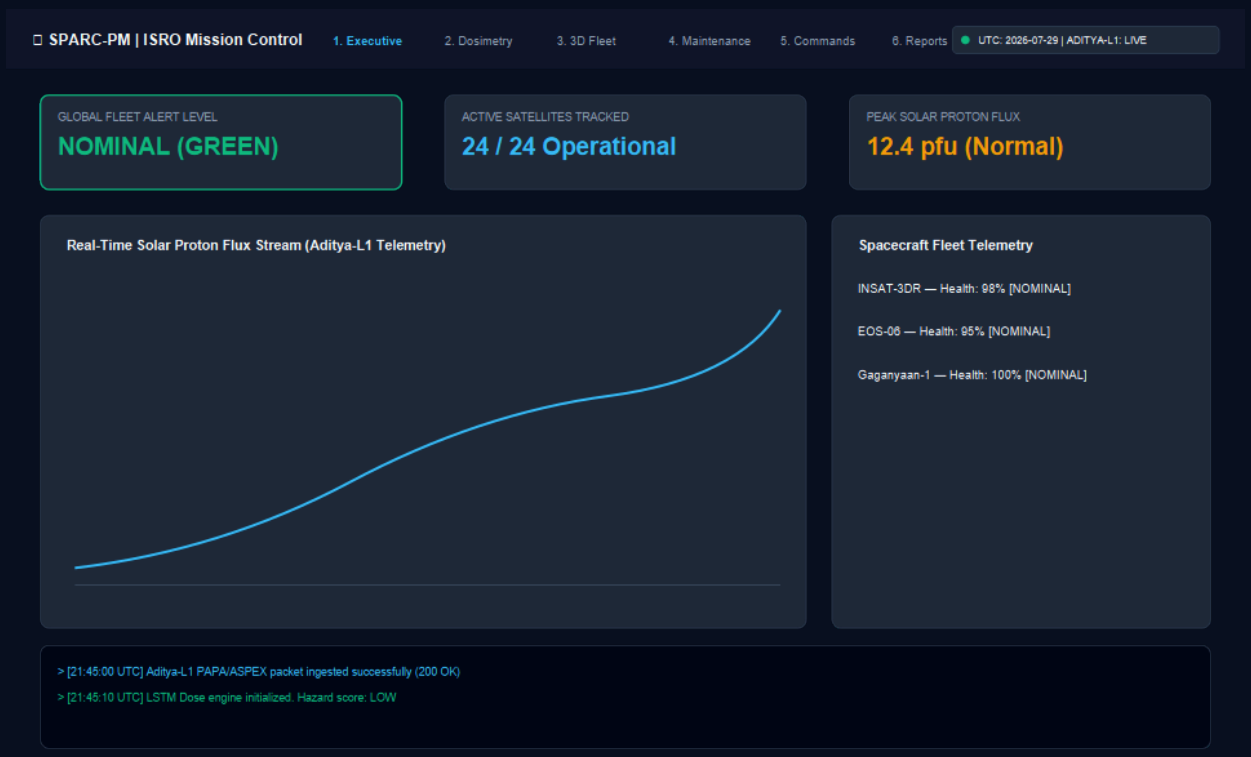
The UI assembles one telemetry packet and delegates computation to engines. This is the main low-coupling seam in the implementation.

```
scheduled_commands = scheduler.schedule_telecommands(
    hazard_report["fleet_hazard_profile"],
    fleet_maintenance
)

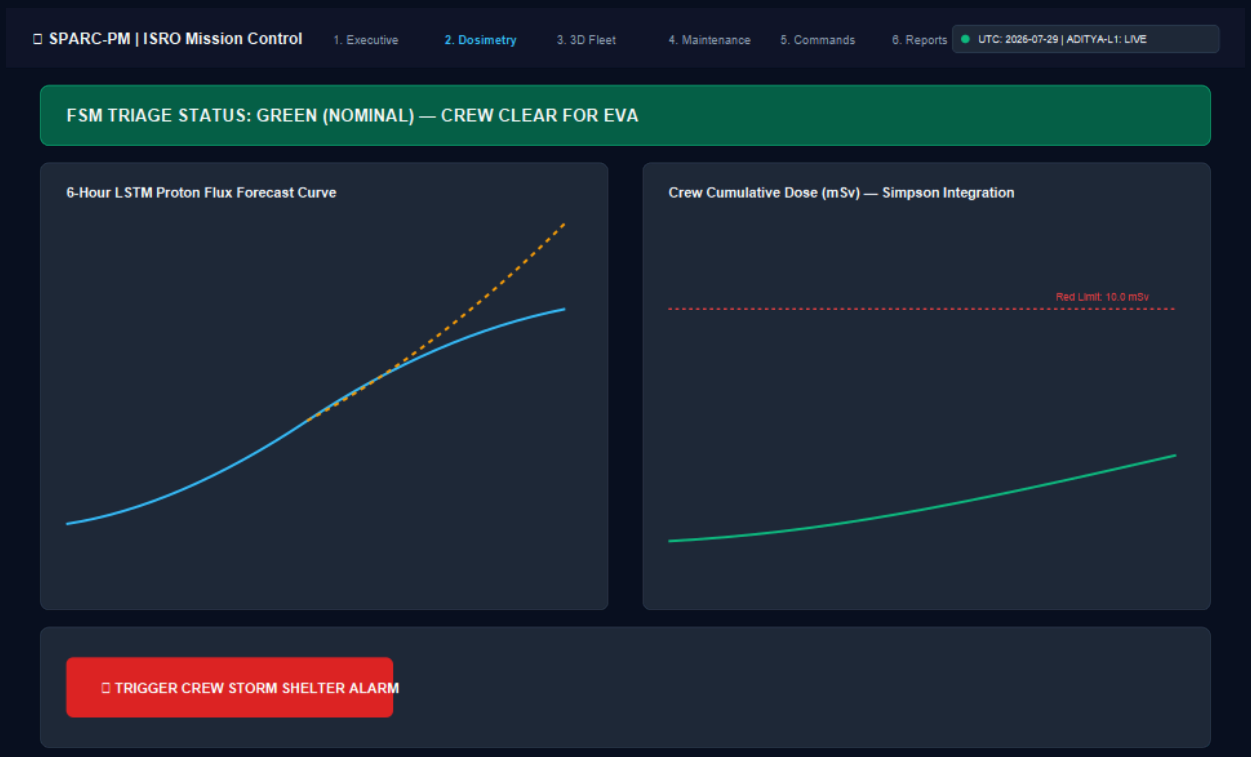
dosimetry_report = dosimetry_eng.forecast_6h_radiation_curve(
    historical_flux_window, sim_speed, sim_bz
)
```

4. User interface design | screens 1-2

Updated wireframe screens generated from the repository's design artifacts.



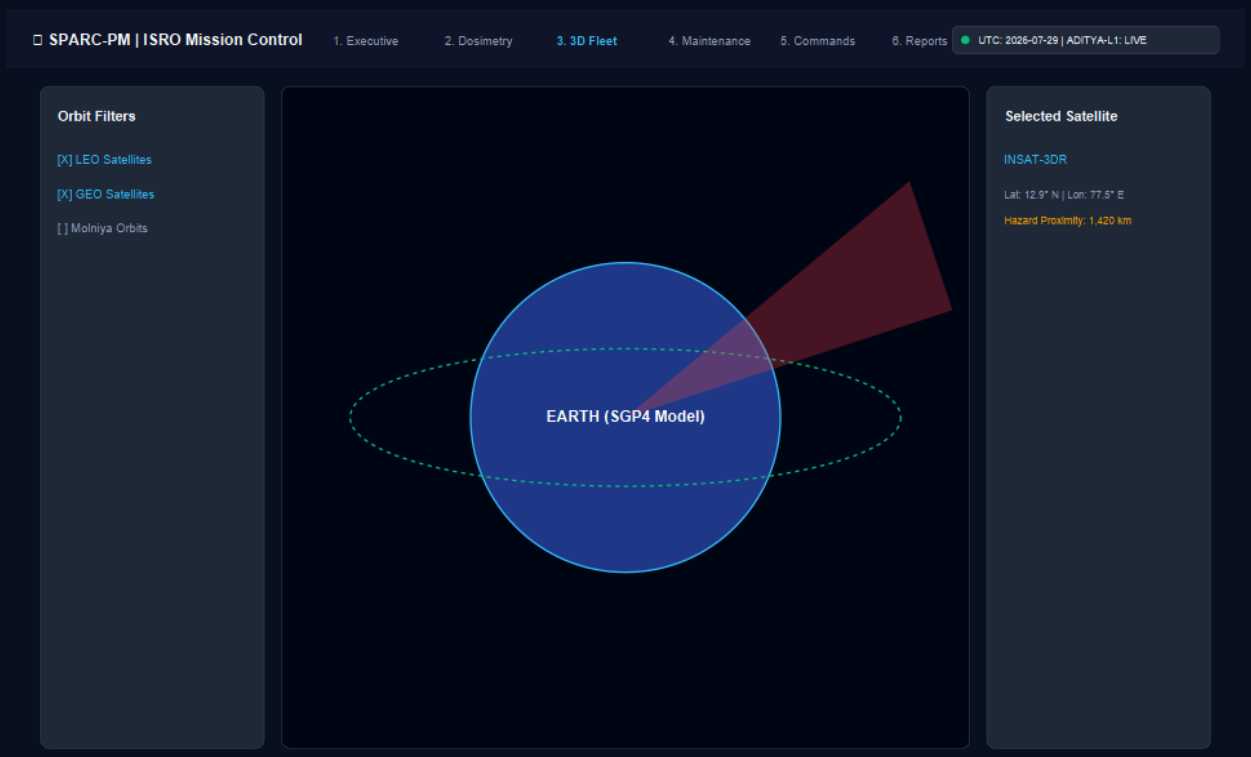
Screen 1: executive overview and triage



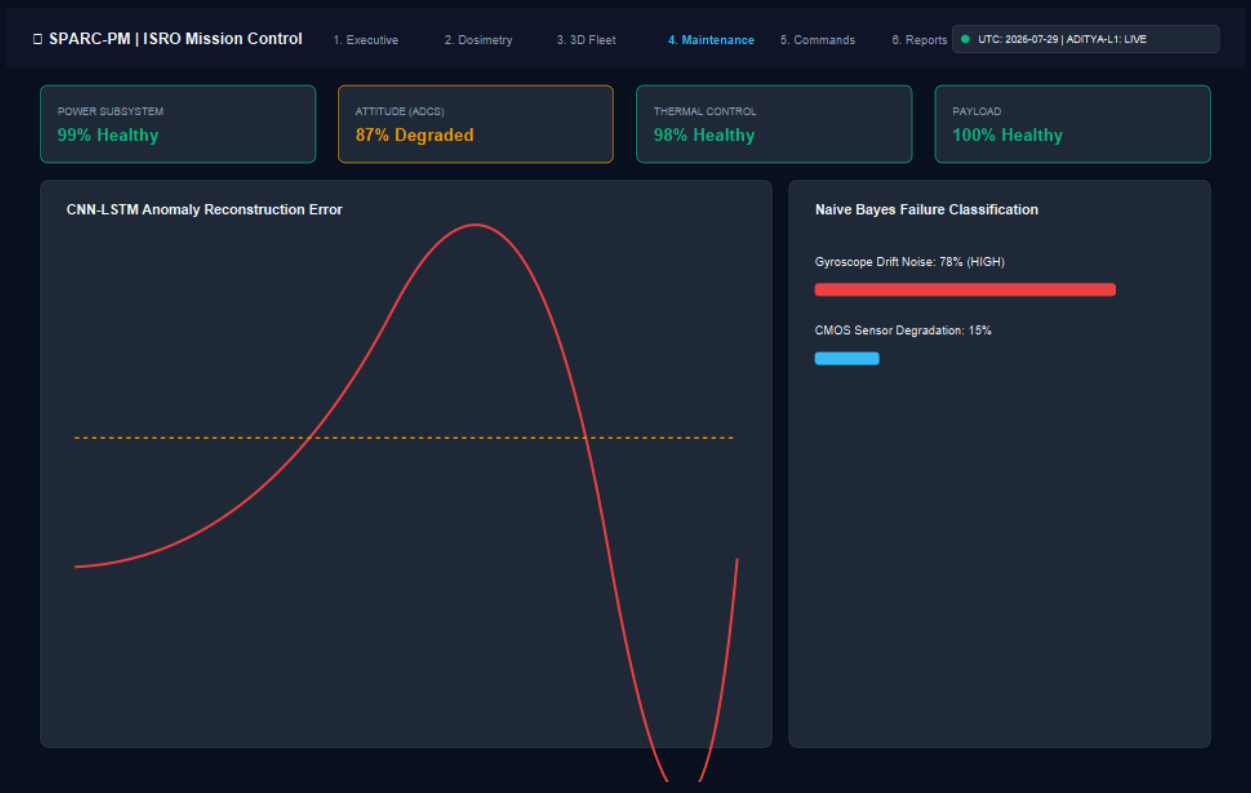
Screen 2: Gaganyaan crew dosimetry

4. User interface design | screens 3-4

Updated wireframe screens generated from the repository's design artifacts.



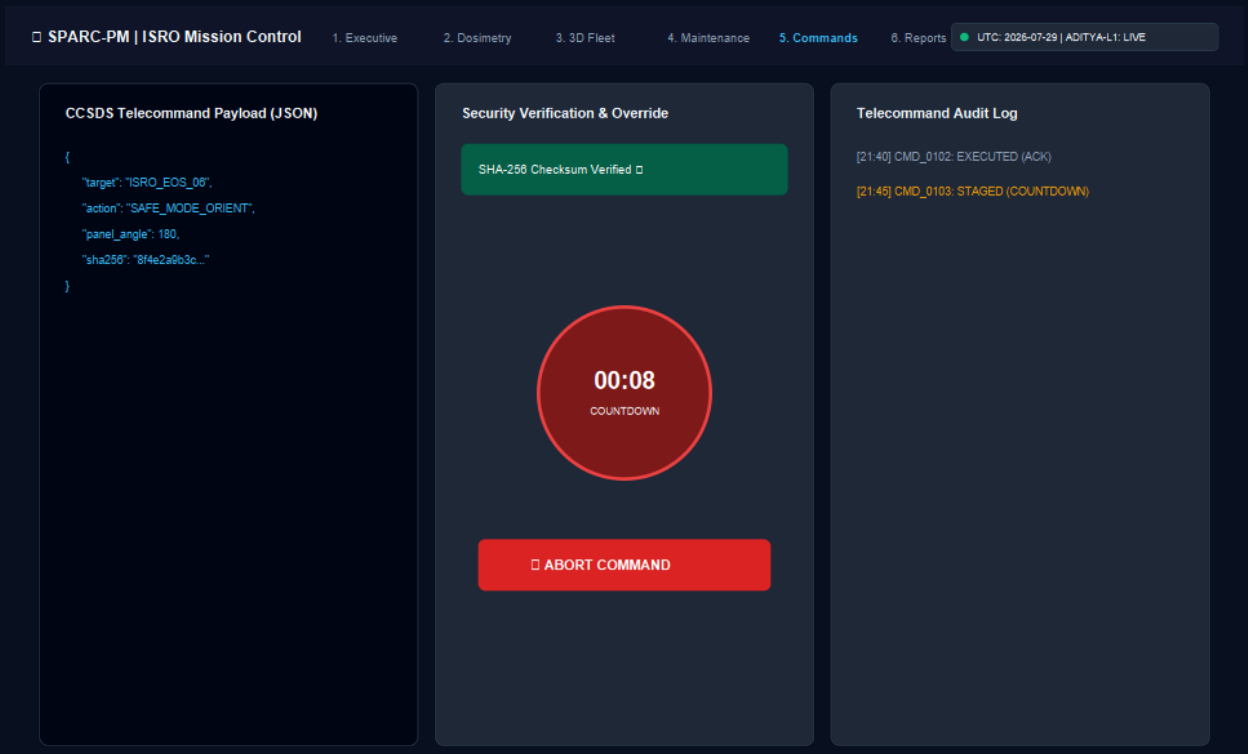
Screen 3: 3D fleet spatial hazard



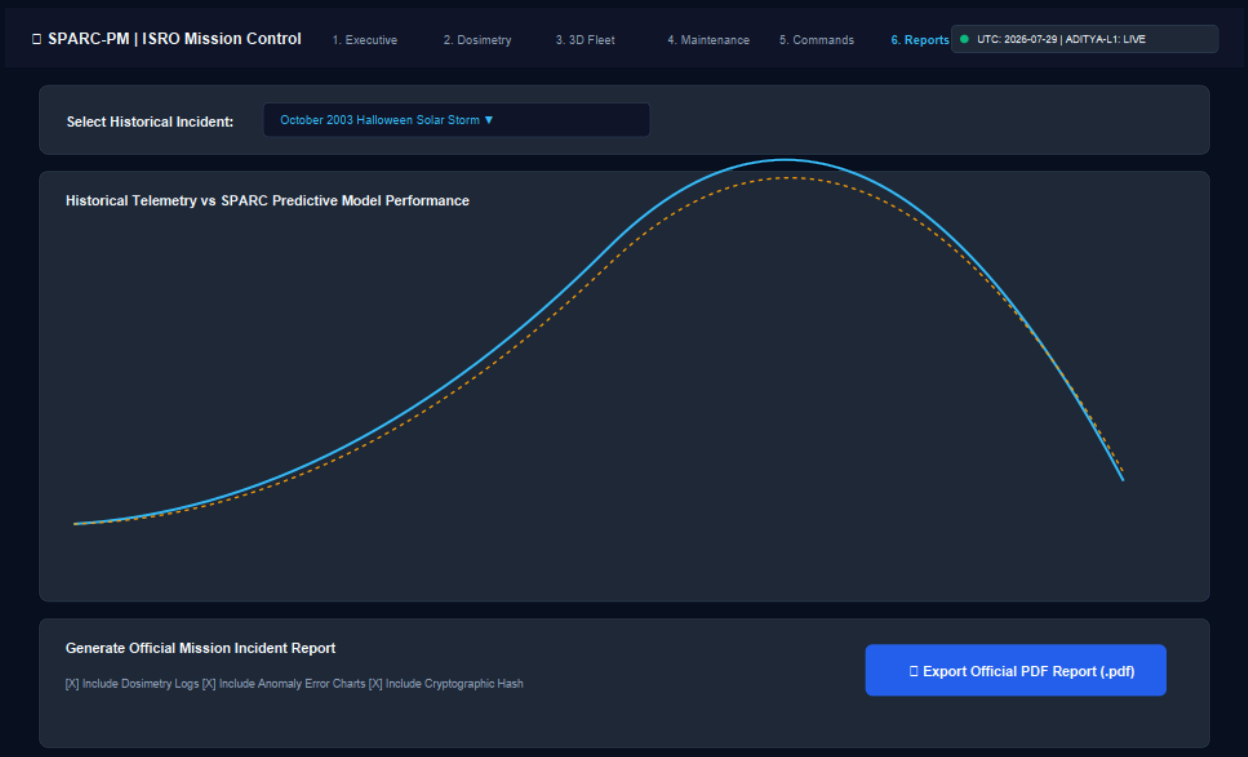
Screen 4: predictive maintenance

4. User interface design | screens 5-6

Updated wireframe screens generated from the repository's design artifacts.



Screen 5: command synthesizer and verification



Screen 6: incident analytics and replay

5. Design decisions and usability rationale

Streamlit as the current shell

It provides a fast, testable mission-control prototype with direct access to Python models and Plotly visualizations. A service/API split remains a future deployment decision.

Six task-oriented screens

The tabs map to the operator workflow: orient, protect crew, inspect fleet, diagnose assets, authorize commands, and review incidents.

Severity color plus plain-language action

Green/yellow/red status is paired with a concrete directive so the interface supports rapid scanning without forcing the operator to interpret a score.

Cached domain engines

Models and fleet state are expensive or stateful resources; caching improves rerun behavior and keeps view code focused on composition.

Signed commands with constraints

A* prioritization is bounded by power, cooldown, and command-count constraints, while HMAC-SHA256 provides an auditable integrity check.

User-friendly interaction patterns

Persistent tab navigation, high-salience status banners, KPI cards, interactive Plotly charts, operator-controlled scenarios, explicit thresholds, and a visible audit/verification path reduce cognitive load during a simulated event.

Known follow-ups

Production hardening should add automated contract tests, externalize the uplink secret, validate the real data-label ground truth, and decide whether the long-term frontend remains Streamlit or moves behind an API.